

C++ Function Wrappers

Jan Wilczek (thinkcell)

Berlin, Sep 28, 2026

Who am I?



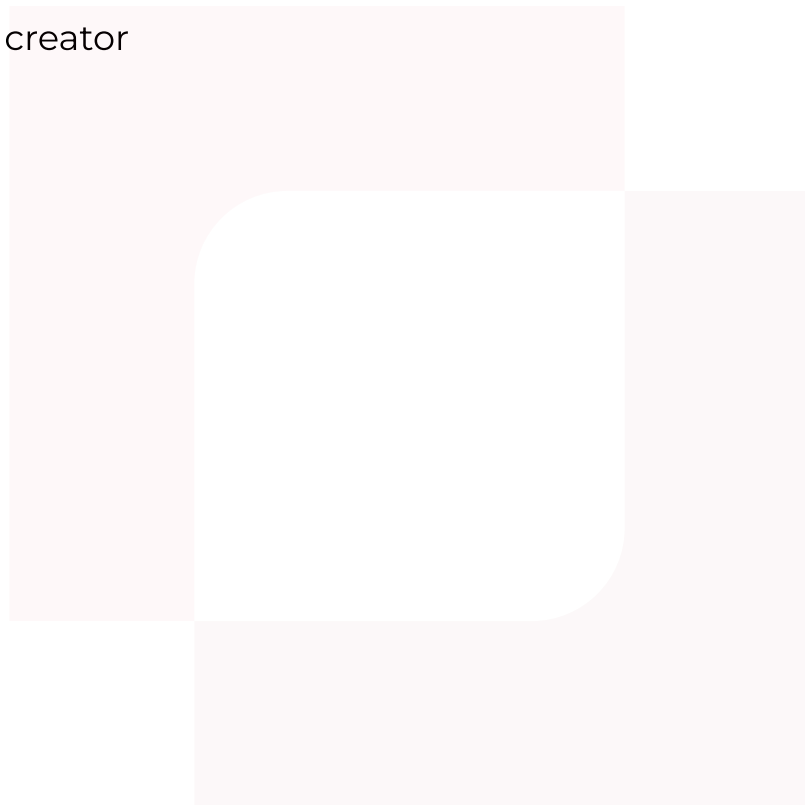
Who am I?

- Jan Wilczek [Yan Vil-check]



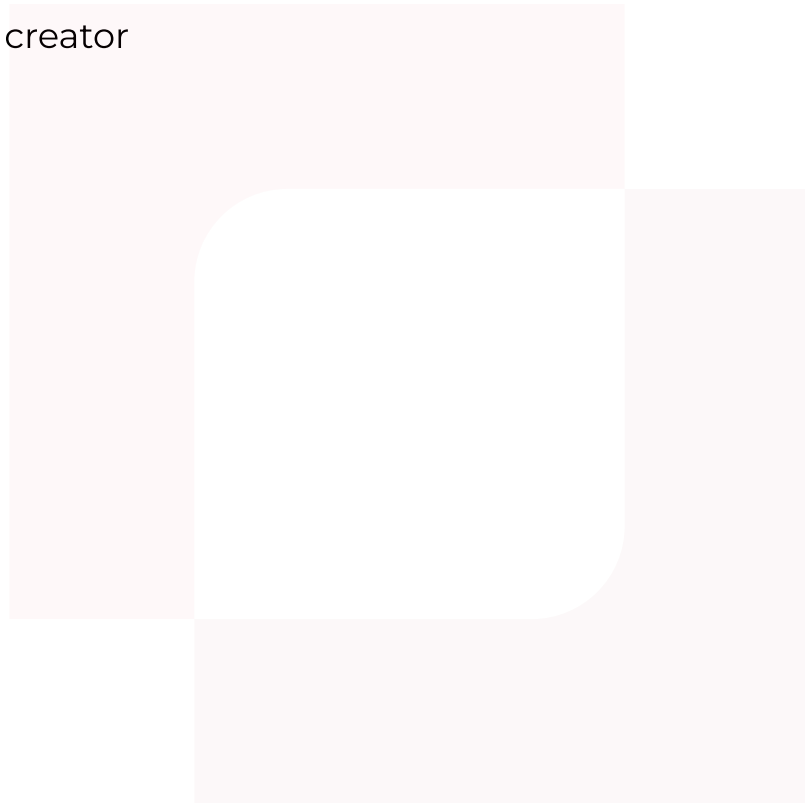
Who am I?

- Jan Wilczek [Yan Vil-check]
- Founder of TheWolfSound.com & online course creator



Who am I?

- Jan Wilczek [Yan Vil-check]
- Founder of TheWolfSound.com & online course creator
- C++ Speaker at think-cell





Access 250+ best-practice templates

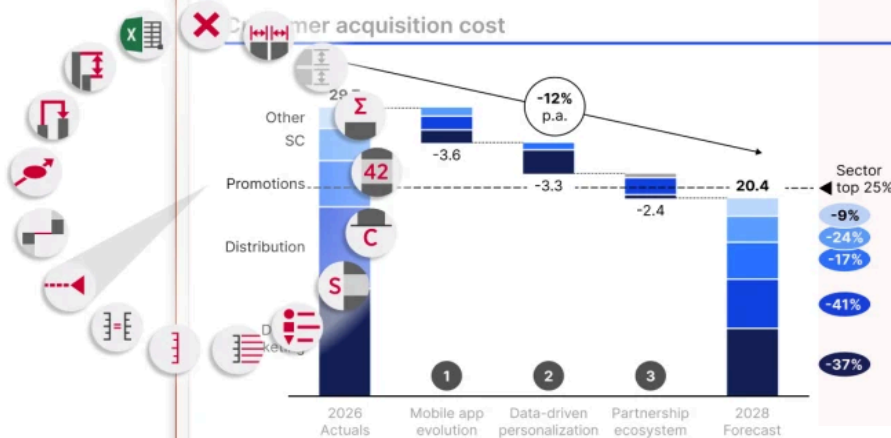


Use AI-powered tools inside PowerPoint and Excel

think-cell



Optimize customer acquisition
cost by -12% p.a. to approach sector benchmarks by 2028



Key projects and funding required



Automate charts with Excel-linking

1. Mobile app evolution
 - Booking engine
 - Self-service purchase, seat selection

	A	B	C	D
1	Category	Digital mark..	Distribution..	Promotions..
2	2026 Actuals (Total)	13	10	5
3	Mobile app evolution	-3.6		
4	Data-driven personal..	-3.3		
5	Partnership ecosyst...	-2.4		
6	2028 Forecast	8	5	3



Create management-level charts and presentations



Fast & Furious



Fast & Furious



Fast & Furious



Fast & Furious



Fast & Furious



Fast & `std::function`



Fast & `std::function`

- `std::function`



Fast & `std::function`

- `std::function`
- `std::move_only_function` (C++23)



Fast & `std::function`

- `std::function`
- `std::move_only_function` (C++23)
- `std::function_ref` (C++26)



Fast & `std::function`

- `std::function`
- `std::move_only_function` (C++23)
- `std::function_ref` (C++26)
- `std::copyable_function` (C++26)

Fast & `std::function`

- `std::function`
- `std::move_only_function` (C++23)
- `std::function_ref` (C++26)
- `std::copyable_function` (C++26)

Contents



Contents

- Need for functions as first-class objects
 - const-correctness
 - copyability
 - movability
 - `noexcept`



Contents

- Need for functions as first-class objects
 - const-correctness
 - copyability
 - movability
 - `noexcept`
- `std::function` & its problems



Contents

- Need for functions as first-class objects
 - const-correctness
 - copyability
 - movability
 - `noexcept`
- `std::function` & its problems
- `std::move_only_function` (C++23) & differences w.r.t. `std::function`

Contents

- Need for functions as first-class objects
 - const-correctness
 - copyability
 - movability
 - `noexcept`
- `std::function` & its problems
- `std::move_only_function` (C++23) & differences w.r.t. `std::function`
- `std::copyable_function` (C++26) & differences w.r.t. `std::move_only_function`

Contents

- Need for functions as first-class objects
 - const-correctness
 - copyability
 - movability
 - `noexcept`
- `std::function` & its problems
- `std::move_only_function` (C++23) & differences w.r.t. `std::function`
- `std::copyable_function` (C++26) & differences w.r.t. `std::move_only_function`
- `std::function_ref` (C++26) & differences

Contents

- Need for functions as first-class objects
 - const-correctness
 - copyability
 - movability
 - `noexcept`
- `std::function` & its problems
- `std::move_only_function` (C++23) & differences w.r.t. `std::function`
- `std::copyable_function` (C++26) & differences w.r.t. `std::move_only_function`
- `std::function_ref` (C++26) & differences
- Will `std::function` be deprecated?

Contents

- Need for functions as first-class objects
 - const-correctness
 - copyability
 - movability
 - `noexcept`
- `std::function` & its problems
- `std::move_only_function` (C++23) & differences w.r.t. `std::function`
- `std::copyable_function` (C++26) & differences w.r.t. `std::move_only_function`
- `std::function_ref` (C++26) & differences
- Will `std::function` be deprecated?
- Pictures of muscled men

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 void doStuff() {  
2     // do stuff  
3 }
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 void doStuff() {  
2     // do stuff  
3     std::println("Stuff done.");  
4 }
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [](std::string_view str) {
9     std::println("Message: {}", str);
10 };
11 doStuff(logger);
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [](std::string_view str) {
9     std::println("Message: {}", str);
10 };
11 doStuff(logger);
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [](std::string_view str) {
9     std::println("Message: {}", str);
10 };
11 doStuff(logger);
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [](std::string_view str) {
9     std::println("Message: {}", str);
10 };
11 doStuff(logger);
```


Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [](std::string_view str) {
9     std::println("Message: {}", str);
10 };
11 doStuff(logger);
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [i = 0](std::string_view str) mutable {
9     std::println("Message {}: {}", i, str);
10    ++i;
11 };
12 doStuff(logger);
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [i = 0](std::string_view str) mutable {
9     std::println("Message {}: {}", i, str);
10    ++i;
11 };
12 doStuff(logger);
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [i = 0](std::string_view str) mutable {
9     std::println("Message {}: {}", i, str);
10    ++i;
11 };
12 doStuff(logger); // ❌
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 using Logger = void (*)(std::string_view);
2
3 void doStuff(Logger log) {
4     // do stuff
5     log("Stuff done.");
6 }
7
8 auto logger = [i = 0](std::string_view str) mutable {
9     std::println("Message {}: {}", i, str);
10    ++i;
11 };
12 doStuff(logger); // ✗
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 void doStuff(std::function<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [i = 0](std::string_view str) mutable {  
7     std::println("Message {}: {}", i, str);  
8     ++i;  
9 };  
10 doStuff(logger); // ✓
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 void doStuff(std::function<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [i = 0](std::string_view str) mutable {  
7     std::println("Message {}: {}", i, str);  
8     ++i;  
9 };  
10 doStuff(logger); // ✓
```

Motivating Example: Logger

<https://godbolt.org/z/MGW3bv4rr>

```
1 void doStuff(std::function<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [i = 0](std::string_view str) mutable {  
7     std::println("Message {}: {}", i, str);  
8     ++i;  
9 };  
10 doStuff(logger); // ✓
```


`std::function`



`std::function`



- C++11

std::function



- C++11
- pass callables without explicit types or templates

std::function



- C++11
- pass callables without explicit types or templates
- functions as first-class objects

std::function



- C++11
- pass callables without explicit types or templates
- functions as first-class objects
- dependency injection (Strategy)

std::function



- C++11
- pass callables without explicit types or templates
- functions as first-class objects
- dependency injection (Strategy)
- callbacks (Observer, GUI, threads, async operations)

std::function



- C++11
- pass callables without explicit types or templates
- functions as first-class objects
- dependency injection (Strategy)
- callbacks (Observer, GUI, threads, async operations)
- deferred execution

std::function



- C++11
- pass callables without explicit types or templates
- functions as first-class objects
- dependency injection (Strategy)
- callbacks (Observer, GUI, threads, async operations)
- deferred execution
- algorithms

std::function



- C++11
- pass callables without explicit types or templates
- functions as first-class objects
- dependency injection (Strategy)
- callbacks (Observer, GUI, threads, async operations)
- deferred execution
- algorithms
- ABI stability

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(std::function<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [i = 0](std::string_view str) mutable {  
7     std::println("Message {}: {}", i, str);  
8     ++i;  
9 };  
10 doStuff(logger);
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(std::function<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [i = 0](std::string_view str) mutable {  
7     std::println("Message {}: {}", i, str);  
8     ++i;  
9 };  
10 doStuff(logger);  
11 doStuff(logger);
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(std::function<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [i = 0](std::string_view str) mutable {  
7     std::println("Message {}: {}", i, str);  
8     ++i;  
9 };  
10 doStuff(logger); // Message 0: Stuff done.  
11 doStuff(logger); // Message 0: Stuff done.
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(std::function<void(std::string_view)> log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = [i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 };
10 doStuff(logger); // Message 0: Stuff done.
11 doStuff(logger); // Message 0: Stuff done.
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(std::function<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [i = 0](std::string_view str) mutable {  
7     std::println("Message {}: {}", i, str);  
8     ++i;  
9 };  
10 doStuff(logger); // Message 0: Stuff done.  
11 doStuff(logger); // Message 0: Stuff done.
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(const std::function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = std::function{[i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 }};
10 doStuff(logger);
11 doStuff(logger);
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(const std::function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = std::function{[i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 }};
10 doStuff(logger);
11 doStuff(logger);
```


std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(const std::function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = std::function{[i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 }};
10 doStuff(logger);
11 doStuff(logger);
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(const std::function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = std::function{[i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 }};
10 doStuff(logger);
11 doStuff(logger);
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(const std::function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = std::function{[i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 }};
10 doStuff(logger); // Message 0: Stuff done.
11 doStuff(logger); // Message 1: Stuff done.
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(const std::function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = std::function{[i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 }};
10 doStuff(logger); // Message 0: Stuff done.
11 doStuff(logger); // Message 1: Stuff done.
```

std::function

<https://godbolt.org/z/nG7ME146d>

```
1 void doStuff(const std::function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = std::function{[i = 0](std::string_view str) mutable {
7     std::println("Message {}: {}", i, str);
8     ++i;
9 }};
10 doStuff(logger); // Message 0: Stuff done.
11 doStuff(logger); // Message 1: Stuff done.
```

std::function: Problem 1 ("constness bug")

<https://godbolt.org/z/vKsd8cPe3>

```
1 auto f = std::function{[i = 0] mutable {  
2     ++i;  
3     std::println("i={}", i);  
4 }};  
5 const auto& fref = f;  
6 fref(); // i=1 ⚠
```

N4159

std::function: Problem 1 ("constness bug")

<https://godbolt.org/z/vKsd8cPe3>

```
1 auto f = std::function{[i = 0] mutable {  
2     ++i;  
3     std::println("i={}", i);  
4 }};  
5 const auto& fref = f;  
6 fref(); // i=1 ⚠
```

N4159

std::function: Problem 1 ("constness bug")

<https://godbolt.org/z/vKsd8cPe3>

```
1 auto f = std::function{[i = 0] mutable {  
2     ++i;  
3     std::println("i={}", i);  
4 }};  
5 const auto& fref = f;  
6 fref(); // i=1 ⚠
```

N4159

std::function: Problem 1 ("constness bug")

<https://godbolt.org/z/vKsd8cPe3>

```
1 auto f = std::function{[i = 0] mutable {  
2     ++i;  
3     std::println("i={}", i);  
4 }};  
5 const auto& fref = f;  
6 fref(); // i=1 ⚠
```

N4159

std::function: Problem 1 ("constness bug")

<https://godbolt.org/z/vKsd8cPe3>

```
1 auto f = std::move_only_function{[i = 0] mutable {  
2     ++i;  
3     std::println("i={}", i);  
4 }};  
5 const auto& fref = f;  
6 fref(); // ❌
```

N4159

"Awesome" logger

<https://godbolt.org/z/xK5bK7vxo>

```
1 namespace awe {
2     struct AwesomeLogger {
3         void log(std::string_view) & { /* ... */ }
4     };
5 }
6
7 void doStuff(std::function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::make_unique<awe::AwesomeLogger>();
13 doStuff(/* ? */);
```

"Awesome" logger

<https://godbolt.org/z/xK5bK7vxo>

```
1 namespace awe {
2 struct AwesomeLogger {
3     void log(std::string_view) & { /* ... */ }
4 };
5 }
6
7 void doStuff(std::function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::make_unique<awe::AwesomeLogger>();
13 doStuff(/* ? */);
```

"Awesome" logger

<https://godbolt.org/z/xK5bK7vx0>

```
1 namespace awe {
2 struct AwesomeLogger {
3     void log(std::string_view) & { /* ... */ }
4 };
5 }
6
7 void doStuff(std::function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::make_unique<awe::AwesomeLogger>();
13 doStuff(/* ? */);
```

"Awesome" logger

<https://godbolt.org/z/xK5bK7vx0>

```
1 namespace awe {
2 struct AwesomeLogger {
3     void log(std::string_view) & { /* ... */ }
4 };
5 }
6
7 void doStuff(std::function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::make_unique<awe::AwesomeLogger>();
13 doStuff(/* ? */);
```

"Awesome" logger

<https://godbolt.org/z/xK5bK7vx0>

```
1 namespace awe {
2 struct AwesomeLogger {
3     void log(std::string_view) & { /* ... */ }
4 };
5 }
6
7 void doStuff(std::function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::make_unique<awe::AwesomeLogger>();
13 doStuff(/* ? */);
```

"Awesome" logger

<https://godbolt.org/z/xK5bK7vx0>

```
1 namespace awe {
2     struct AwesomeLogger {
3         void log(std::string_view) & { /* ... */ }
4     };
5 }
6
7 void doStuff(std::function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::function{
13     [logger = std::make_unique<awe::AwesomeLogger>()] (std::string_view str) {
14         logger->log(str);
15     }
16 };
17 doStuff(logger);
```


"Awesome" logger

<https://godbolt.org/z/xK5bK7vx0>

```
1 namespace awe {
2     struct AwesomeLogger {
3         void log(std::string_view) & { /* ... */ }
4     };
5 }
6
7 void doStuff(std::function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::function{
13     [logger = std::make_unique<awe::AwesomeLogger>()] (std::string_view str) {
14         logger->log(str);
15     }
16 }; // ✗
17 doStuff(logger);
```

"Awesome" logger

<https://godbolt.org/z/xK5bK7vxo>

```
1 namespace awe {
2 struct AwesomeLogger {
3     void log(std::string_view) & { /* ... */ }
4 };
5 }
6
7 void doStuff(std::move_only_function<void(std::string_view)>& log) {
8     // do stuff
9     log("Stuff done.");
10 }
11
12 auto logger = std::move_only_function<void(std::string_view)>>{
13     [logger = std::make_unique<awe::AwesomeLogger>()] (std::string_view str) {
14         logger->log(str);
15     }
16 }; // ✅
17 doStuff(logger);
```

```
std::move_only_function
```



```
std::move_only_function
```



- C++23 (P0288)

`std::move_only_function`



- C++23 (P0288)
- non-copyable

`std::move_only_function`



- C++23 (P0288)
- non-copyable
- allows move-only callables

`std::move_only_function`



- C++23 (P0288)
- non-copyable
- allows move-only callables
- allows copyable callables

`std::move_only_function`



- C++23 (P0288)
- non-copyable
- allows move-only callables
- allows copyable callables
- const-correct

`std::move_only_function`



- C++23 (P0288)
- non-copyable
- allows move-only callables
- allows copyable callables
- const-correct
- fixes a few more problems of `std::function` ...

`std::move_only_function`



- C++23 (P0288)
- non-copyable
- allows move-only callables
- allows copyable callables
- const-correct
- fixes a few more problems of `std::function` ...
- Available in github.com/think-cell/think-cell-library

std::function: Problem 2

<https://godbolt.org/z/WzsfjGrev>

```
1 auto f = std::function<void(void)>{[i = 0] {  
2     std::println("i={}", i);  
3 }};  
4 const auto& fref = f;  
5 fref(); // ✓
```

std::function: Problem 2

<https://godbolt.org/z/WzsfjGrev>

```
1 auto f = std::move_only_function<void(void)>{[i = 0] {  
2     std::println("i={}", i);  
3 }};  
4 const auto& fref = f;  
5 fref(); // ✗
```

std::function: Problem 2

<https://godbolt.org/z/WzsfjGrev>

```
1 auto f = std::move_only_function<void(void) const>{[i = 0] {  
2     std::println("i={}", i);  
3 }};  
4 const auto& fref = f;  
5 fref(); // ✓
```

std::function: Problem 3

<https://godbolt.org/z/P5EhvsoMv>

```
1 std::function<void(void)> get_f();  
2 // ...  
3 auto f = get_f();  
4 f(); // does this throw?
```

std::function: Problem 3

<https://godbolt.org/z/P5EhvsoMv>

```
1 std::move_only_function<void(void) noexcept> get_f();  
2 // ...  
3 auto f = get_f();  
4 f(); // if throws, program terminates
```

std::function: Problem 3

<https://godbolt.org/z/1zx8vKK6W>

```
1 auto f = std::move_only_function<void(void) const noexcept>{[] {  
2     std::println("I don't throw");  
3 }}; // ❌
```


std::function: Problem 3

<https://godbolt.org/z/1zx8vKK6W>

```
1 auto f = std::move_only_function<void(void) const noexcept>{[] noexcept {  
2     std::println("I don't throw");  
3 }}; // ✓
```

Limitation of `std::move_only_function`

```
1 auto f = std::move_only_function<void(void) const noexcept> {[i = 0] noexcept {  
2     std::println("i={}", i);  
3 }};  
4 const auto g = f; // ❌
```

Limitation of `std::move_only_function`

```
1 auto f = std::copyable_function<void(void) const noexcept>{[i = 0] noexcept {  
2     std::println("i={}", i);  
3 }};  
4 const auto g = f; // ✓
```

```
std::copyable_function
```



↔ `std::move_only_function` +

`std::copyable_function`



↔ `std::move_only_function` +

- copy constructor

`std::copyable_function`



↔ `std::move_only_function` +

- copy constructor
- copy assignment operator

`std::copyable_function`



↔ `std::move_only_function` +

- copy constructor
- copy assignment operator
- callables must be copy-constructible

`std::copyable_function`



↔ `std::move_only_function` +

- copy constructor
- copy assignment operator
- callables must be copy-constructible
- C++26 (P2548)

Thread-safe logger

<https://godbolt.org/z/4YrE485sj>

```
1 void doStuff(std::move_only_function<void(std::string_view)>& log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [logger = std::make_unique<awe::AwesomeLogger>()] (std::string_view str) {  
7     logger->log(str);  
8 };  
9 doStuff(logger);
```

Thread-safe logger

<https://godbolt.org/z/4YrE485sj>

```
1 void doStuff(std::move_only_function<void(std::string_view)>& log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [logger = std::make_unique<awe::AwesomeLogger>()] (std::string_view str) {  
7     logger->log(str);  
8 };  
9 doStuff(logger);
```

Thread-safe logger

<https://godbolt.org/z/4YrE485sj>

```
1 void doStuff(std::move_only_function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = [logger = std::make_unique<awe::AwesomeLogger>(), mutex = std::mutex{}] (std::string_view str) {
7     std::lock_guard lock{mutex};
8     logger->log(str);
9 };
10 doStuff(logger);
```

Thread-safe logger

<https://godbolt.org/z/4YrE485sj>

```
1 void doStuff(std::move_only_function<void(std::string_view)>& log) {
2     // do stuff
3     log("Stuff done.");
4 }
5
6 auto logger = [logger = std::make_unique<awe::AwesomeLogger>(), mutex = std::mutex{}] (std::string_view str) {
7     std::lock_guard lock{mutex};
8     logger->log(str);
9 };
10 doStuff(logger);
```

Thread-safe logger

<https://godbolt.org/z/4YrE485sj>

```
1 void doStuff(std::move_only_function<void(std::string_view)>& log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [logger = std::make_unique<awe::AwesomeLogger>(), mutex = std::mutex{}] (std::string_view str) {  
7     std::lock_guard lock{mutex};  
8     logger->log(str);  
9 };  
10 doStuff(logger); // ❌
```

Thread-safe logger

<https://godbolt.org/z/4YrE485sj>

```
1 void doStuff(std::move_only_function<void(std::string_view)>& log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [logger = std::make_unique<awe::AwesomeLogger>(), mutex = std::mutex{}] (std::string_view str) {  
7     std::lock_guard lock{mutex};  
8     logger->log(str);  
9 };  
10 doStuff(logger); // ❌
```

Thread-safe logger

<https://godbolt.org/z/4YrE485sj>

```
1 void doStuff(std::function_ref<void(std::string_view)> log) {  
2     // do stuff  
3     log("Stuff done.");  
4 }  
5  
6 auto logger = [logger = std::make_unique<awe::AwesomeLogger>(), mutex = std::mutex{}] (std::string_view str) {  
7     std::lock_guard lock{mutex};  
8     logger->log(str);  
9 };  
10 doStuff(logger); // ✓
```

```
std::function_ref
```



`std::function_ref`



- C++26 (P0792)

std::function_ref



- C++26 ([P0792](#))
- Non-owning callable wrapper

`std::function_ref`



- C++26 (P0792)
- Non-owning callable wrapper
- Does for functions the same job as `std::string_view` for `std::string`

`std::function_ref`



- C++26 (P0792)
- Non-owning callable wrapper
- Does for functions the same job as `std::string_view` for `std::string`
- Similar features to `std::move_only_function`
 - const correctness
 - `const` / `noexcept` / `ref` qualifiers
 - more

std::function_ref



- C++26 (P0792)
- Non-owning callable wrapper
- Does for functions the same job as `std::string_view` for `std::string`
- Similar features to `std::move_only_function`
 - const correctness
 - `const` / `noexcept` / `ref` qualifiers
 - more
- Available in github.com/think-cell/think-cell-library

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function<void(std::string_view)> logger)
3         : m_logger(std::move(logger)) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function<void(std::string_view)> m_logger;
12     int m_i;
13 };
```

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function<void(std::string_view)> logger)
3         : m_logger(std::move(logger)) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function<void(std::string_view)> m_logger;
12     int m_i;
13 };
```

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function<void(std::string_view)> logger)
3         : m_logger(std::move(logger)) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function<void(std::string_view)> m_logger;
12     int m_i;
13 };
```


Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function<void(std::string_view)> logger)
3         : m_logger(std::move(logger)) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function<void(std::string_view)> m_logger;
12     int m_i;
13 };
```

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function<void(std::string_view)> logger)
3         : m_logger(std::move(logger)) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function<void(std::string_view)> m_logger;
12     int m_i;
13 };
```

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function_ref<void(std::string_view)> logger)
3         : m_logger(logger) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function_ref<void(std::string_view)> m_logger;
12     int m_i;
13 };
```

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function_ref<void(std::string_view)> logger)
3         : m_logger(logger) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function_ref<void(std::string_view)> m_logger;
12     int m_i;
13 };
14
15 auto logger = CountingLogger{[name = "foo"] (std::string_view str) {
16     std::println("{}: {}", name, str);
17 }};
18 doStuff(logger);
```

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function_ref<void(std::string_view)> logger)
3         : m_logger(logger) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function_ref<void(std::string_view)> m_logger;
12     int m_i;
13 };
14
15 auto logger = CountingLogger{[name = "foo"] (std::string_view str) {
16     std::println("{}: {}", name, str);
17 }};
18 doStuff(logger); // disaster 🤖
```

Composite logger

<https://godbolt.org/z/dxav63a7K>

```
1 struct CountingLogger {
2     CountingLogger(std::function_ref<void(std::string_view)> logger)
3         : m_logger(logger) {}
4
5     void operator()(std::string_view str) {
6         ++m_i;
7         m_logger(str);
8     }
9
10 private:
11     std::function_ref<void(std::string_view)> m_logger;
12     int m_i;
13 };
14
15 auto logger = CountingLogger{[name = "foo"] (std::string_view str) {
16     std::println("[{}]: {}", name, str);
17 }};
18 doStuff(logger); // disaster 🧟
```

Further differences from `std::function`

Ref-qualifiers

<https://godbolt.org/z/8KbMErcf>

```
1  auto f = std::function<void(void)>{ // cannot have & or && within <>
2      [] { std::println("Hello, Ref/Ref Ref/Val!"); } };
3  f();
4  std::move(f)();
5  std::function<void(void)>{
6      [] { std::println("Hello, Ref/Ref Ref/Val!"); } }();
```


Ref-qualifiers

<https://godbolt.org/z/8KbMErcf>

```
1  auto f = std::function<void(void)>{ // cannot have & or && within <>
2      [] { std::println("Hello, Ref/Ref Ref/Val!"); } };
3  f();
4  std::move(f)();
5  std::function<void(void)>{
6      [] { std::println("Hello, Ref/Ref Ref/Val!"); } }();
```

Ref-qualifiers

<https://godbolt.org/z/8KbMErcf>

```
1  auto f = std::function<void(void)>{ // cannot have & or && within <>
2      [] { std::println("Hello, Ref/Ref Ref/Val!"); } };
3  f();
4  std::move(f)();
5  std::function<void(void)>{
6      [] { std::println("Hello, Ref/Ref Ref/Val!"); } }();
```

Ref-qualifiers

<https://godbolt.org/z/8KbMErcf>

```
1  auto f = std::function<void(void)>{ // cannot have & or && within <>
2      [] { std::println("Hello, Ref/Ref Ref/Val!"); } };
3  f();
4  std::move(f)();
5  std::function<void(void)>{
6      [] { std::println("Hello, Ref/Ref Ref/Val!"); } }();
```

Ref-qualifiers

<https://godbolt.org/z/8KbMErccf>

```
1  auto f = std::function<void(void)>{ // cannot have & or && within <>
2      [] { std::println("Hello, Ref/Ref Ref/Val!"); } };
3  f();
4  std::move(f)();
5  std::function<void(void)>{
6      [] { std::println("Hello, Ref/Ref Ref/Val!"); } }();
```

Ref-qualifiers

<https://godbolt.org/z/8KbMErcf>

```
1  std::move_only_function<void(void) &>{[] {
2      std::println("Hello, Ref!");
3  }}(); // ✗
4
5  auto f_ref = std::move_only_function<void(void) &>{
6      [] { std::println("Hello, Ref!"); }};
7  f_ref();
8
9  auto f_ref_ref = std::move_only_function<void(void) &&>{
10     [] { std::println("Hello, Ref Ref!"); }};
11  f_ref_ref(); // ✗
12  std::move(f_ref_ref)()
```

Ref-qualifiers

<https://godbolt.org/z/8KbMErcf>

```
1  std::move_only_function<void(void) &>{[] {
2      std::println("Hello, Ref!");
3  }}(); // ✗
4
5  auto f_ref = std::move_only_function<void(void) &>{
6      [] { std::println("Hello, Ref!"); };
7  f_ref();
8
9  auto f_ref_ref = std::move_only_function<void(void) &&>{
10     [] { std::println("Hello, Ref Ref!"); };
11     f_ref_ref(); // ✗
12     std::move(f_ref_ref)()
```

Ref-qualifiers

<https://godbolt.org/z/8KbMErcf>

```
1  std::move_only_function<void(void) &>{[] {
2      std::println("Hello, Ref!");
3  }}(); // ✗
4
5  auto f_ref = std::move_only_function<void(void) &>{
6      [] { std::println("Hello, Ref!"); };
7  f_ref();
8
9  auto f_ref_ref = std::move_only_function<void(void) &&>{
10     [] { std::println("Hello, Ref Ref!"); };
11     f_ref_ref(); // ✗
12     std::move(f_ref_ref)()
```

No `target_type()` and `target()` accessors (requested by users and implementors)

<https://godbolt.org/z/6rxGvfGEW>

```
1 void foo() {  
2     std::println("Hello, Type!");  
3 }  
4  
5 auto f = std::function<void(void)>{foo};  
6 std::println("{} ", f.target_type().name()); // prints "PFvvE"  
7  
8 if (void (*const* ptr)() = f.target<void(*)()>()) {  
9     std::println("Guessed it!");  
10 }
```


No `target_type()` and `target()` accessors (requested by users and implementors)

<https://godbolt.org/z/6rxGvfGEW>

```
1 void foo() {  
2     std::println("Hello, Type!");  
3 }  
4  
5 auto f = std::function<void(void)>{foo};  
6 std::println("{} ", f.target_type().name()); // prints "PFvvE"  
7  
8 if (void (*const* ptr)() = f.target<void(*)>()) {  
9     std::println("Guessed it!");  
10 }
```

No `target_type()` and `target()` accessors (requested by users and implementors)

<https://godbolt.org/z/6rxGvfGEW>

```
1 void foo() {  
2     std::println("Hello, Type!");  
3 }  
4  
5 auto f = std::function<void(void)>{foo};  
6 std::println("{} ", f.target_type().name()); // prints "PFvvE"  
7  
8 if (void (*const* ptr)() = f.target<void(*)>()) {  
9     std::println("Guessed it!");  
10 }
```

"Invocation has strong preconditions"

<https://godbolt.org/z/8b5fMbaoe>

```
1  std::function<void(void)>{}(); // std::bad_function_call
2  std::move_only_function<void(void)>{}(); // UB
3  std::copyable_function<void(void)>{}(); // UB
4  std::function_ref<void(void)>{}(); // ❌
```

"Invocation has strong preconditions"

<https://godbolt.org/z/8b5fMbaoe>

```
1  std::function<void(void)>{}(); // std::bad_function_call
2  std::move_only_function<void(void)>{}(); // UB
3  std::copyable_function<void(void)>{}(); // UB
4  std::function_ref<void(void)>{}(); // ✗
```

"Invocation has strong preconditions"

<https://godbolt.org/z/8b5fMbaoe>

```
1  std::function<void(void)>{}(); // std::bad_function_call
2  std::move_only_function<void(void)>{}(); // UB
3  std::copyable_function<void(void)>{}(); // UB
4  std::function_ref<void(void)>{}(); // ✗
```

"Invocation has strong preconditions"

<https://godbolt.org/z/8b5fMbaoe>

```
1  std::function<void(void)>{}(); // std::bad_function_call
2  std::move_only_function<void(void)>{}(); // UB
3  std::copyable_function<void(void)>{}(); // UB
4  std::function_ref<void(void)>{}(); // ❌
```

"Invocation has strong preconditions"

Why? 🤔



"Invocation has strong preconditions"

Why? 🤔

https://www.reddit.com/r/cpp_questions/s/yxX2MXa4Yy

"Invocation has strong preconditions"

Why? 🤔



"Invocation has strong preconditions"

Why? 🤔

- Invoking an empty `std::function` is a bug

"Invocation has strong preconditions"

Why? 🤔

- Invoking an empty `std::function` is a bug
- Implementations can now assert on this

"Invocation has strong preconditions"

Why? 🤔

- Invoking an empty `std::function` is a bug
- Implementations can now assert on this
- "Don't pay for what you don't use"

"Invocation has strong preconditions"

Why? 🤔

- Invoking an empty `std::function` is a bug
- Implementations can now assert on this
- "Don't pay for what you don't use"
- Exceptionless environments

"Invocation has strong preconditions"

Why? 🤔

- Invoking an empty `std::function` is a bug
- Implementations can now assert on this
- "Don't pay for what you don't use"
- Exceptionless environments
- `noexcept` propagation

```
1  std::throwing_move_only_function<void(void) noexcept> f;  
2  f(); // throws despite noexcept because empty  
3  f = [] noexcept { /* ... */ }; // assign
```

I guess Java/C#/Go wasn't getting enough backend projects and Rust wasn't getting enough low level work. Gotta take the opportunity to make C++ just a little more unsafe and make sure even the most up-to-date C++ still has a mountain of gotchas baked in. Don't worry; I'm sure there will be a safety profile for that later. (Yeah right.)

The language is too big to die, but not for lack of trying.

The C++ language development strategy at this is to pretty much fiddle while Rome burns basically.

I guess Java/C#/Go wasn't getting enough backend projects and Rust wasn't getting enough low level work. Gotta take the opportunity to make C++ just a little more unsafe and make sure even the most up-to-date C++ still has a mountain of gotchas baked in. Don't worry; I'm sure there will be a safety profile for that later. (Yeah right.)

The language is too big to die, but not for lack of trying.

The C++ language development strategy at this is to pretty much fiddle while Rome burns basically.

Conversions

From \ To	function	move_only_function	copyable_function	function_ref
function		✓	✓	✓
move_only_function	✗		✗	✓
copyable_function	✓	✓		✓
function_ref	✓	✓	✓	

Summary: `std::function` problems



Summary: `std::function` problems

- can call a non-const callable via a const ref



Summary: `std::function` problems

- can call a non-const callable via a const ref
- disallows move-only callables



Summary: `std::function` problems

- can call a non-const callable via a const ref
- disallows move-only callables
- disallows non-movable, non-copyable callables

Summary: `std::function` problems

- can call a non-const callable via a const ref
- disallows move-only callables
- disallows non-movable, non-copyable callables
- does not propagate `const`, `noexcept`, `&`, or `&&` to `operator()`

Summary: `std::function` problems

- can call a non-const callable via a const ref
- disallows move-only callables
- disallows non-movable, non-copyable callables
- does not propagate `const`, `noexcept`, `&`, or `&&` to `operator()`
- throws `std::bad_function_call`

Will `std::function` **be deprecated?**

Probably not 🙄

Probably not 🙄

But there is a proposal for it: P2721

Summary



Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks



Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied

Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied
- use `std::function_ref` (C++26) if you don't need to store the function wrapper or the callable cannot be moved

Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied
- use `std::function_ref` (C++26) if you don't need to store the function wrapper or the callable cannot be moved
- use `std::copyable_function` for a copyable function wrapper

Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied
- use `std::function_ref` (C++26) if you don't need to store the function wrapper or the callable cannot be moved
- use `std::copyable_function` for a copyable function wrapper
- `std::copyable_function` $\iff$ `std::move_only_function` +
 - copy constructor
 - copy assignment operator
 - callables must be copy-constructible

Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied
- use `std::function_ref` (C++26) if you don't need to store the function wrapper or the callable cannot be moved
- use `std::copyable_function` for a copyable function wrapper
- `std::copyable_function` $\iff$ `std::move_only_function` +
 - copy constructor
 - copy assignment operator
 - callables must be copy-constructible
- avoid `std::function`

Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied
- use `std::function_ref` (C++26) if you don't need to store the function wrapper or the callable cannot be moved
- use `std::copyable_function` for a copyable function wrapper
- `std::copyable_function` $\iff$ `std::move_only_function` +
 - copy constructor
 - copy assignment operator
 - callables must be copy-constructible
- avoid `std::function`
- never call an empty function wrapper (UB)

Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied
- use `std::function_ref` (C++26) if you don't need to store the function wrapper or the callable cannot be moved
- use `std::copyable_function` for a copyable function wrapper
- `std::copyable_function` $\iff$ `std::move_only_function` +
 - copy constructor
 - copy assignment operator
 - callables must be copy-constructible
- avoid `std::function`
- never call an empty function wrapper (UB)
- check out think-cell-library: github.com/think-cell/think-cell-library

Summary

- function pointers and STL function wrappers are great solutions for dependency inversion and callbacks
- use `std::move_only_function` (C++23) if your function wrapper doesn't have to be copied or the callable cannot be copied
- use `std::function_ref` (C++26) if you don't need to store the function wrapper or the callable cannot be moved
- use `std::copyable_function` for a copyable function wrapper
- `std::copyable_function` $\iff$ `std::move_only_function` +
 - copy constructor
 - copy assignment operator
 - callables must be copy-constructible
- avoid `std::function`
- never call an empty function wrapper (UB)
- check out think-cell-library: github.com/think-cell/think-cell-library

